index

Symbols

[BOS] (beginning of sequence) token 32
[EOS] (end of sequence) token 32
[PAD] (padding) token 32
@ operator 261
%timeit command 282
<|endoftext|> token 34
<|unk|> tokens 29–31, 34
== comparison operator 277

Numerics

04_preference-tuning-with-dpo folder 247
124M parameter 161
355M parameter 227

A

AdamW optimizer 148, 294
AI (artificial intelligence) 252
allowed_max_length 224, 233, 309
Alpaca dataset 233, 296
alpha scaling factor 328
architectures, transformer 7–10
argmax function 134, 152–155, 190, 277
arXiv 248
assign utility function 165
attention mechanisms
  causal 74–82
  coding 50, 54
  implementing self-attention with trainable weights 64–74
  multi-head attention 82–91
  problem with modeling long sequences 52
  self-attention mechanism 55–64
attention scores 57
attention weights, computing step by step 65–70
attn_scores 71
autograd engine 264
automatic differentiation 263–265
  engine 252
  partial derivatives and gradients 263
autoregressive model 13
Axolotl 249

B

backpropagation 137
.backward() method 112, 318
Bahdanau attention mechanism 54
base model 7
batch normalization layers 276
batch_size 233
BERT (bidirectional encoder representations from transformers) 8
BPE (byte pair encoding) 32–35

C

calc_accuracy_loader function 192
calc_loss_batch function 145, 193–194
calc_loss_loader function 144, 194
calculating, training and validation 140, 142
CausalAttention class 80–81, 86, 90
  module 83–84
  object 86

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

causal attention mask 190
causal attention mechanism 74–82
cfg dictionary 115, 119
classification
  fine-tuning
    categories of 170
    preparing dataset 172–175
  fine-tuning for
    adding classification head 183–190
    calculating classification loss and accuracy 190–194
    supervised data 195–200
    using LLM as spam classifier 200
  tasks 7
classify_review function 200
clip_grad_norm_ function 317
clipping, gradient 317
code for data loaders 301
coding
  attention mechanisms 54
  GPT model 117–122
collate function 211
computation graphs 261
compute_accuracy function 277–278
computing gradients 258
connections, shortcut 109–113
context, adding special tokens 29–32
context_length 47, 95
context vectors 57, 64, 85
conversational performance 236
converting tokens into token IDs 24–29
cosine decay 313, 316
create_dataloader_v1 function 39
cross_entropy function 138–139
CUDA_VISIBLE_DEVICES environment variable 286
custom_collate_draft_1 215
custom_collate_draft_2 218
custom_collate_fn function 224, 308

D

data, sampling with sliding window 35–41
DataFrame 173
data list 207, 209
DataLoader class 38, 211, 224, 270–272
data loaders 175–181
  code for 301
  creating for instruction dataset 224–226
  efficient 270–274
Dataset class 38, 177, 270–272, 274
datasets
  downloading 207
  preparing 324
  utilizing large 10
DDP (DistributedDataParallel) strategy 282
ddp_setup function 286
decode method 27, 33–34
decoder 52
decoding strategies to control randomness 151–159
  modifying text generation function 157
  temperature scaling 152–155
  top-k sampling 155
deep learning 253
  library 252
destroy_process_group function 284
device variable 224
dim parameter 101–102
DistributedDataParallel class 284
DistributedSampler 283–284
Dolma: An Open Corpus of Three Trillion Tokens for LLM Pretraining Research (Soldaini et al.) 11
dot products 58
d_out argument 90, 301
download_and_load_gpt2 function 161, 163, 182
drop_last parameter 273
dropout
  defined 78
  layers 276
drop_rate 95
.dtype attribute 259
DummyGPTClass 98
DummyGPTModel 95, 97–98, 117
DummyLayerNorm 97, 99, 117
  placeholder 100
DummyTransformerBlock 97, 117

E

emb_dim 95
Embedding layer 161
embedding size 46
emergent behavior 14
encode method 27, 33, 37
encoder 52
encoding word positions 43–47
entry dictionary 209
eps variable 103
.eval() mode 126
eval_iter value 200
evaluate_model function 147–148, 196

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

F

feedforward layer 267
FeedForward module 107–108, 113
feed forward network, implementing with GELU activations 105–109
find_highest_gradient function 318
fine-tuning
  categories of 170
  creating data loaders for instruction dataset 224–226
  evaluating fine-tuned LLMs 238–247
  extracting and saving responses 233–238
  for classification 169
    adding classification head 183–190
    calculating classification loss and accuracy 190–194
    data loaders 175–181
    fine-tuning model on supervised data 195–200
    initializing model with pretrained weights 181
    preparing dataset 172–175
    using LLM as spam classifier 200
  instruction data 230–233
  instruction fine-tuning, overview 205
  LLMs, to follow instructions 204
  organizing data into training batches 211–223
  supervised instruction fine-tuning, preparing dataset for 207–211
FineWeb Dataset 295
first_batch variable 39
format_input function 209–210, 242, 307
forward method 97, 109, 267, 330
foundation model 7
fully connected layer 267
functools standard library 224

G

GELU (Gaussian error linear unit) 105, 107, 293
  activation function 104, 111
GenAI (generative AI) 3
generate_and_print_sample function 147–148, 151, 154
generate function 157, 159, 167, 228, 234–235, 237, 305
generate_model_scores function 246
generate_simple function 157, 159
generate_text_simple function 125–126, 131–132, 134, 148, 151–153
generative text models, evaluating 129
__getitem__ method 271
Google Colab 257
GPT-2 94
  model 230
  tokenizer 176
gpt2-medium355M-sft.pth file 238
GPT-3 11, 94
GPT-4 239
GPT_CONFIG_124M dictionary 95, 97, 107, 116–117, 120, 127, 130
GPTDatasetV1 class 38–39
gpt_download.py Python module 161
GPT (Generative Pre-trained Transformer) 8, 18, 93
  architecture 12–14
  coding 117–122
  coding architecture 93–99
  implementing feed forward network with GELU activations 105–109
  implementing from scratch, shortcut connections 109–113
  implementing from scratch to generate text 92, 122
  implementing model from scratch 99–105, 113–116
GPTModel 119, 121–122, 133, 146, 182, 330
  class 122, 130, 182, 326
  code 141
  implementation 166
  instance 131, 159, 164–167
GPUs (graphics processing units), optimizing training performance with 279–288
.grad attribute 318
grad_fn value 268
grad function 264
gradient clipping 313, 317
gradients 263
greedy decoding 125, 152

I

information leakage 76
__init__ constructor 71, 81, 119, 266–267, 271
initializing model 326
initial_lr 314
init_process_group function 284
input_chunk tensor 38
input_embeddings 47
'input' object 208
instruction data, fine-tuning LLMs on 230–233
instruction dataset 205
InstructionDataset class 212, 224, 308

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

instruction fine-tuning 7, 170, 322
instruction following, creating data loaders for instruction dataset 224–226
'instruction' object 208
instruction–response pairs 207
loading pretrained LLMs 226–229
overview 205

K

keepdim parameter 101

L

LayerNorm 103, 115, 117, 119
layer normalization 99–105
learning rate warmup 313–314
__len__ method 271
LIMA dataset 296
Linear layers 95, 107, 329–330, 332–333
Linear layer weights 330
LinearWithLoRA layer 330–331, 333
LitGPT 249
LLama 2 model 141
Llama 3 model 238
llama.cpp library 238
LLMs (large language models) 17–18
applications of 4
building and using 5–7, 14
coding architecture 93–99
coding attention mechanisms, causal attention mechanism 74–82
fine-tuning 230–233, 238–247, 295
fine-tuning for classification 183–194, 200
implementing GPT model, implementing feed forward network with GELU activations 105–109
instruction fine-tuning, loading pretrained LLMs 226–229
overview of 1–4
pretraining 132, 140, 142, 146–151, 159
training function 313, 319–321
training loop, gradient clipping 317
transformer architecture 7–10
utilizing large datasets 10
working with text data, word embeddings 18–20
loading, pretrained weights from OpenAI 160–167
load_state_dict method 160
load_weights_into_gpt function 165–166, 182
logistic regression loss function 293
logits tensor 139
LoRALayer class 329–330
LoRA (low-rank adaptation) 247, 322
parameter-efficient fine-tuning 324, 326
loss.backward() function 112
losses 140, 142
loss metric 132
lr (learning rate) 275

M

machine learning 253
Machine Learning Q and AI (Raschka) 290
macOS 282
main function 286
masked attention 74
.matmul method 261
matrices 258–261
max_length 38, 141, 178, 306
minbpe repository 291
model_configs table 164
model.eval() function 160
model.named_parameters() function 112
model.parameters() method 129
model_response 238
model.train() setting 276
model weights, loading and saving in PyTorch 159
Module base class 265
mps device 224
mp.spawn() call 286
multi-head attention 80, 82–91
implementing with weight splits 86–91
stacking multiple single-head attention layers 82–85
MultiHeadAttention class 86–87, 90–91, 292
MultiHeadAttentionWrapper class 83–87, 90
multilayer neural networks, implementing 265–269
multinomial function 153–155
multiprocessing.spawn function 284
multiprocessing submodule 284

N

NeuralNetwork model 284
neural networks
implementing feed forward network with GELU activations 105–109
implementing multilayer neural networks 265–269
NEW_CONFIG dictionary 164
n_heads 95

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

nn.Linear layers 72
nn.Module 71, 97
numel() method 120
num_heads dimension 88
num_tokens dimension 88

O

Ollama application 238, 241
Ollama Llama 3 method 309
ollama run command 242
ollama run llama3 command 240–241
ollama serve command 239–242
OLMo 294
one-dimensional tensor (vector) 259
OpenAI, loading pretrained weights from 160–167
OpenAI's GPT-3 Language Model: A Technical Overview 293
optimizer.step() method 276
optimizer.zero_grad() method 276
out_head 97
output layer nodes 183
'output' object 208

P

parameter-efficient fine-tuning 322
  LoRA (low-rank adaptation) 322
  preparing dataset 324
parameters 129
  calculating 302
params dictionary 162, 164–165
partial derivatives 263
partial function 224
peak_lr 314
perplexity 139
Phi-3 model 297
PHUDGE model 297
pip installer 33
plot_losses function 232
plot_values function 199
pos_embeddings 47
Post-LayerNorm 115
preference fine-tuning 298
Pre-LayerNorm 115
pretokenizes 212
pretrained weights, initializing model with 181
pretraining 7
  calculating text generation loss 132
  calculating training and validation set losses 140, 142
  decoding strategies to control randomness 151–159
  loading and saving model weights in PyTorch 159
  loading pretrained weights from OpenAI 160–167
  on unlabeled data 128
  training LLMs 146–151
  using GPT to generate text 130
print_gradients function 112
print_sampled_tokens function 155, 304
print statement 24
Prometheus model 297
prompt styles 209
.pth extension 159
Python version 254
PyTorch
  and Torch 256
  automatic differentiation 263–265
  computation graphs 261
  data loaders 210
  dataset objects 325
  efficient data loaders 270–274
  implementing multilayer neural networks 265–269
  installing 254–257
  loading and saving model weights in 159
  optimizing training performance with GPUs 279–288
  overview 251–257
  saving and loading models 278
  training loops 274–278
  understanding tensors 258–261
  with a NumPy-like API 258

Q

qkv_bias 95
Q query matrix 88
query_llama function 243
query_model function 242–243

R

random_split function 175
rank argument 286
raw text 6
register_buffer 81
re library 22
ReLU (rectified linear unit) 100, 105
.replace() method 235
replace_linear_with_lora function 330, 332

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

.reshape method 260–261
re.split command 22
responses, extracting and saving 233–238
retrieval-augmented generation 19
r/LocalLLaMA subreddit 248
RMSNorm 292
RNNs (recurrent neural networks) 52

S

saving and loading models 278
scalars 258–261
scaled dot-product attention 64
scaled_dot_product function 292
scale parameter 103
sci_mode parameter 102
SelfAttention class 90
self-attention mechanism 55–64
  computing attention weights for all input tokens 61–64
  implementing with trainable weights 64–74
  without trainable weights 56–61
SelfAttention_v1 class 71, 73
SelfAttention_v2 class 73
self.out_proj layer 90
self.register_buffer() call 81
self.use_shortcut attribute 111
Sequential class 267
set_printoptions method 277
settings dictionary 162, 164
SGD (stochastic gradient descent) 275
.shape attribute 260, 271
shift parameter 103
shortcut connections 109–113
SimpleTokenizerV1 class 27
SimpleTokenizerV2 class 29, 31, 33
single-head attention, stacking multiple layers 82–85
sliding window 35–41
softmax function 269, 276
softmax_naive function 60
SpamDataset class 176, 178
spawn function 286
special context tokens 29–32
state_dict 160, 279
stride setting 39
strip() function 229
supervised data, fine-tuning model on 195–200
supervised instruction fine-tuning 205
  preparing dataset for 207–211
supervised learning 253
SwiGLU (Swish-gated linear unit) 105

T

target_chunk tensor 38
targets tensor 139
temperature scaling 151–152, 154–155
tensor2d 259
tensor3d 259
Tensor class 258
tensor library 252
tensors 258–261
  common tensor operations 260
  scalars, vectors, matrices, and tensors 258–261
  tensor data types 259
  three-dimensional tensor 259
  two-dimensional tensor 259
test_data set 246
test_loader 272
test_set dictionary 237–238
text completion 205
text data 17
  adding special context tokens 29–32
  converting tokens into token IDs 24–29
  creating token embeddings 42–43
  encoding word positions 43–47
  sliding window 35–41
  tokenization, byte pair encoding 33–35
  word embeddings 18–20
text_data 314
text generation 122
  using GPT to generate text 130
text generation function, modifying 157
text generation loss 132
text_to_token_ids function 131
tiktoken package 176, 178
.T method 261
.to() method 259, 280
token_embedding_layer 46–47
token embeddings 42–43
token IDs 24–29
token_ids_to_text function 131
tokenization, byte pair encoding 33–35
tokenizing text 21–24
top-k sampling 151, 155–156
torch.argmax function 125
torchaudio library 255
torch.manual_seed(123) 272
torch.nn.Linear layers 267
torch.no_grad() context manager 269
torch.save function 159
torch.sum method 277
torch.tensor function 258
torchvision library 255

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

total_loss variable 145
ToyDataset class 271
tqdm progress bar utility 242
train_classifier_simple function 197, 200
train_data subset 143
training, optimizing performance with GPUs 279–288
  PyTorch computations on GPU devices 279
  selecting available GPUs on multi-GPU machine 286–288
  single-GPU training 280
  training with multiple GPUs 282–288
training batches, organizing data into 211–223
training function 319–321
  enhancing 313
  modified 319–321
training loops 274–278
  cosine decay 316
  gradient clipping 317
  learning rate warmup 314
train_loader 272
train_model_simple function 147, 149, 160, 195
train_ratio 142
train_simple_function 305
transformer architecture 3, 7–10, 55
TransformerBlock class 115
transformer blocks 93, 185
  connecting attention and linear layers in 113–116
.transpose method 87
tril function 75

U

UltraChat dataset 297
unbiased parameter 103
unlabeled data, decoding strategies to control randomness 151–159

V

val_data subset 143
variable-length inputs 142
vectors 258–261
.view method 87
vocab_size 95
v vector 317

W

.weight attribute 129, 161
weight_decay parameter 200
weight parameters 66, 129
weights
  initializing model with pretrained weights 181
  loading pretrained weights from OpenAI 160–167
weight splits 86–91
W_k matrix 65, 71
Word2Vec 19
word embeddings 18–20
word positions, encoding 43–47
W_q matrix 65, 71, 88
W_v matrix 65, 71

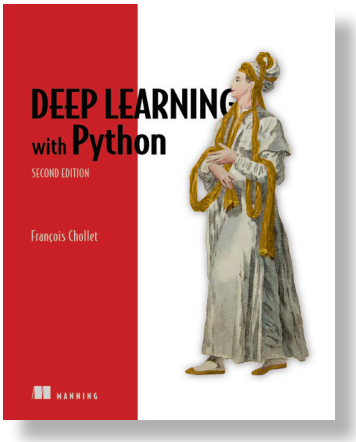
X

X training example 268

Z

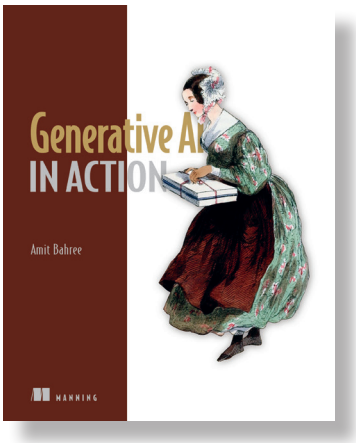
zero-dimensional tensor (scalar) 259

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>



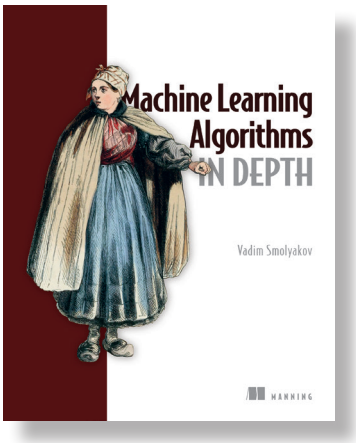
Deep Learning with Python, Second Edition
by Francois Chollet

ISBN 9781617296864
504 pages, $59.99
October 2021



Generative AI in Action
by Amit Bahree

ISBN 9781633436947
469 pages (estimated), $59.99
October 2024 (estimated)



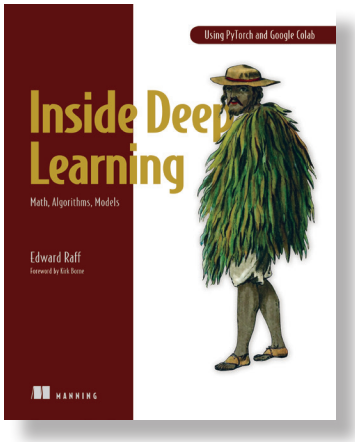
Machine Learning Algorithms in Depth
by Vadim Smolyakov

ISBN 9781633439214
328 pages, $79.99
July 2024

For ordering information, go to www.manning.com

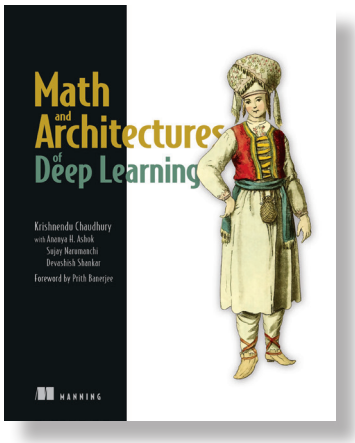
Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

RELATED MANNING TITLES



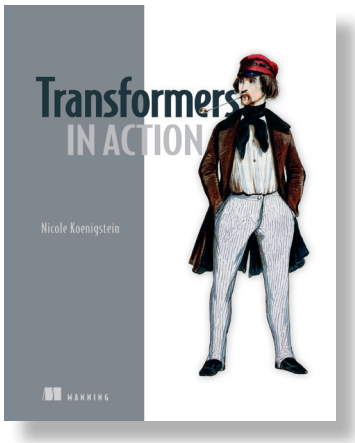
Inside Deep Learning
by Edward Raff
Foreword by Kirk Borne

ISBN 9781617298639
600 pages, $59.99
April 2022



Math and Architectures of Deep Learning
by Krishnendu Chaudhury
with Ananya H. Ashok, Sujay Narumanchi,
Devashish Shankar
Foreword by Prith Banerjee

ISBN 9781617296482
552 pages, $69.99
April 2024



Transformers in Action
by Nicole Koenigstein

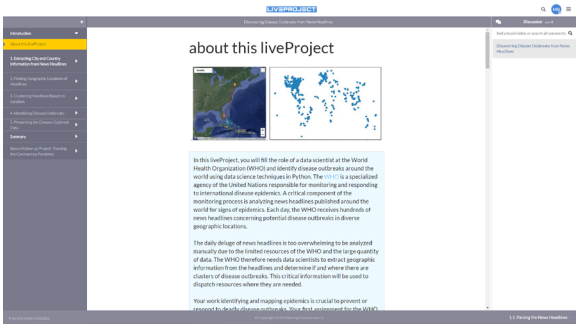
ISBN 9781633437883
393 pages (estimated), $59.99
February 2025 (estimated)

For ordering information, go to www.manning.com

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

MANNING

LIVEPROJECT



Hands-on projects for learning your way

liveProjects are an exciting way to develop your skills that's just like learning on the job.

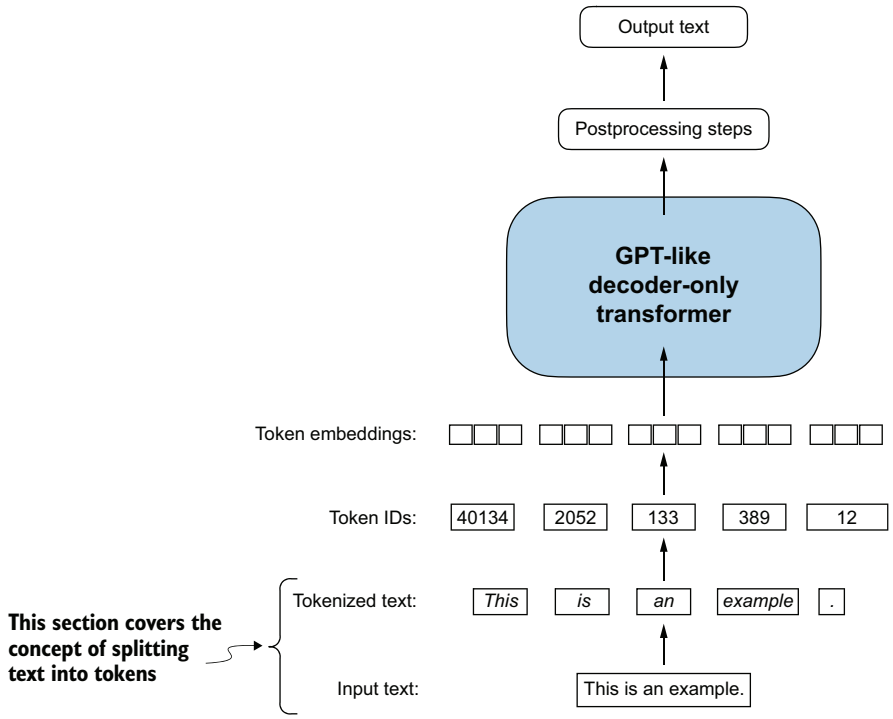
In a Manning liveProject, you tackle a real-world IT challenge and work out your own solutions. To make sure you succeed, you'll get 90 days of full and unlimited access to a hand-picked list of Manning book and video resources.

Here's how liveProject works:

- **Achievable milestones.** Each project is broken down into steps and sections so you can keep track of your progress.
- **Collaboration and advice.** Work with other liveProject participants through chat, working groups, and peer project reviews.
- **Compare your results.** See how your work shapes up against an expert implementation by the liveProject's creator.
- **Everything you need to succeed.** Datasets and carefully selected learning resources come bundled with every liveProject.
- **Build your portfolio.** All liveProjects teach skills that are in demand from industry. When you're finished, you'll have the satisfaction that comes with success and a real project to add to your portfolio.

Explore dozens of data, development, and cloud engineering liveProjects at www.manning.com!

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>



A view of the text processing steps in the context of an LLM. The process starts with input text, which is broken down into tokens and then converted into numerical token IDs. These IDs are linked to token embeddings that serve as the input for the GPT model. The model processes these embeddings and generates output text. Finally, the output undergoes postprocessing steps to produce the final text. This flow illustrates the basic operations of tokenization, embedding, transformation, and postprocessing in a GPT model that is implemented from the ground up in this book.

Licensed to Christopher Funderburk <cmfunderburk@gmail.com>

AI

BUILD A Large Language Model (FROM SCRATCH)

Sebastian Raschka

Physicist Richard P. Feynman reportedly said, "I don't understand anything I can't build." Based on this same powerful principle, bestselling author Sebastian Raschka guides you step by step as you build a GPT-style LLM that you can run on your laptop. This is an engaging book that covers each stage of the process, from planning and coding to training and fine-tuning.

Build a Large Language Model (From Scratch) is a practical and eminently-satisfying hands-on journey into the foundations of generative AI. Without relying on any existing LLM libraries, you'll code a base model, evolve it into a text classifier, and ultimately create a chatbot that can follow your conversational instructions. And you'll really understand it because you built it yourself!

What's Inside

- Plan and code an LLM comparable to GPT-2
- Load pretrained weights
- Construct a complete training pipeline
- Fine-tune your LLM for text classification
- Develop LLMs that follow human instructions

Readers need intermediate Python skills and some knowledge of machine learning. The LLM you create will run on any modern laptop and can optionally utilize GPUs.

Sebastian Raschka is a Staff Research Engineer at Lightning AI, where he works on LLM research and develops open-source software.

The technical editor on this book was David Caswell.

For print book owners, all ebook formats are free:
https://www.manning.com/freebook

MANNING

"Truly inspirational! It motivates you to put your new skills into action."

—Benjamin Muskalla
Senior Engineer, GitHub

"The most understandable and comprehensive explanation of language models yet! Its unique and practical teaching style achieves a level of understanding you can't get any other way."

—Cameron Wolfe
Senior Scientist, Netflix

"Sebastian combines deep knowledge with practical engineering skills and a knack for making complex ideas simple. This is the guide you need!"

—Chip Huyen, author of *Designing Machine Learning Systems* and *AI Engineering*

"Definitive, up-to-date coverage. Highly recommended!"

—Dr. Vahid Mirjalili, Senior Data Scientist, FM Global

Free eBook
See first page

ISBN-13: 978-1-63343-716-6

90000
9 781633 437166